

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: Ciencias de la Computación
	Carrera: Ingeniería de Software Asignatura: Análisis y Diseño de Software NRC: 30724
	Apellidos y nombres de los estudiantes: Díaz Stiven Leiton Jose Manosalvas Gabriel

ANTECEDENTES

1. El Condominio Residencial "La Primavera" presenta actualmente dificultades en la gestión administrativa y financiera debido al uso de procesos manuales para el control de expensas.
2. Se identifica una falta de transparencia y lentitud en la comunicación entre la administración y los copropietarios al momento de reportar y validar pagos.
3. El proyecto AliGest surge como una solución tecnológica para centralizar la información, automatizar el cálculo de moras (12%) y mejorar la trazabilidad de los recursos

OBJETIVO

1. Modelar los casos de uso de Nivel 0 del sistema AliGest utilizando la herramienta PlantUML para definir las interacciones principales entre los actores y el sistema.
2. Garantizar la trazabilidad entre los requisitos funcionales generales y los criterios de aceptación mediante una matriz RF-CU.

DESARROLLO

GUÍA 1 - ANÁLISIS DE CASOS DE USO (CRUD DE PAGOS DE EXPENSAS)

1. REQUISITOS FUNCIONALES

De acuerdo con las reglas de negocio del sistema AliGest establecidas en el documento SRS, la vista de Nivel 0 abarca los cuatro grandes bloques funcionales que orquestan la plataforma. En este enfoque general, se establecen las bases estructurales: el control de acceso (Administrar Sistema), el manejo del directorio de residentes (Gestionar Copropietarios), el

motor financiero (Implementar Pagos) y el motor de mensajería automática (Enviar Notificaciones).

Código	Requisito funcional de nivel 0	Actor principal	Caso de uso asociado
RF-01	El sistema debe gestionar el acceso, autenticación mediante credenciales, recuperación de contraseña y configuración de roles.	Administrador, Copropietario	Administrar Sistema
RF-02	El sistema debe permitir administrar la información de los copropietarios (incluyendo importación masiva) y generar reportes basados en roles.	Administrador, Copropietario	Gestionar Copropietarios
RF-03	El sistema debe gestionar el proceso completo de registro, validación y control de pagos, incluyendo el cálculo automático del recargo del 12% por mora.	Copropietario, Administrador	Implementar Pagos
RF-04	El sistema debe enviar automáticamente mensajes de WhatsApp a los copropietarios para notificar aprobación, rechazo o sanción por mora.	Sistema	Enviar Notificaciones

2. DIAGRAMA DE CASOS DE USO

A continuación, se presenta el código PlantUML:

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam shadowing false

actor "Administrador" as Admin
actor "Copropietario" as Copro
actor "API WhatsApp" as API_WA <<Sistema Externo>>

rectangle "Módulos Principales" {
    usecase "RF-01: Administrar Sistema" as UC1
    usecase "RF-02: Gestionar Copropietarios" as UC2
    usecase "RF-03: Implementar Pagos" as UC3
    usecase "RF-04: Enviar Notificaciones" as UC4
}

Admin --> UC1
Admin --> UC2
Admin --> UC3
Copro --> UC3

UC3 ..> UC4 : <<include>>
UC4 --> API_WA

note bottom of UC3
    Incluye validación y
    sanción por mora (12%)
end note
@enduml
```

Resultado

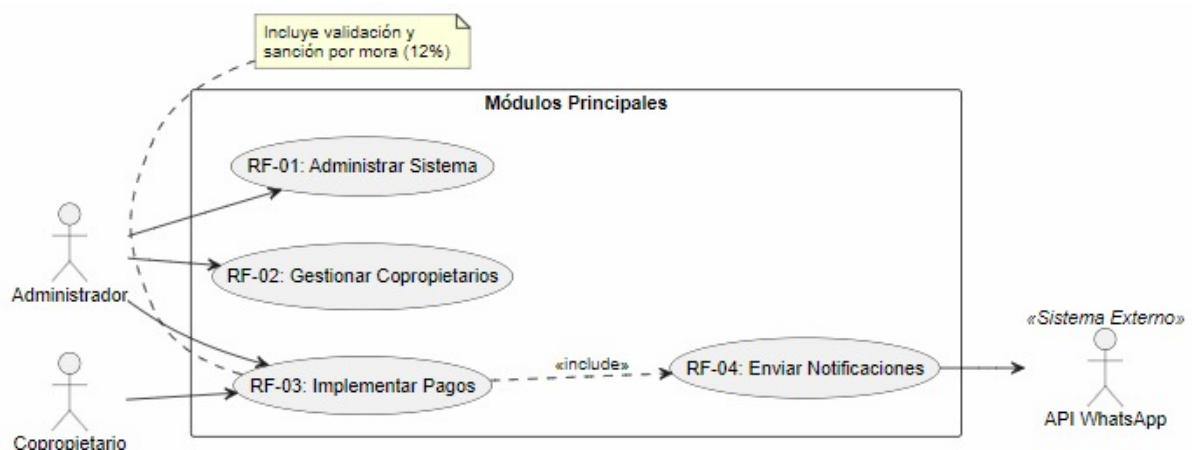


Diagrama Puente

A continuación, se presenta el código PlantUML:

@startuml

left to right direction

actor "Administrador" as Admin

actor "Copropietario" as Copro

boundary "Interfaz del Sistema\nCondominio La Primavera" as UI

control "Control de Gestión" as Ctrl

entity "Módulos del Sistema" as Mod

database "Base de Datos" as DB

collections "API WhatsApp" as API_WA

Admin --> UI : administra sistema,\ngestiona copropietarios y pagos

Copro --> UI : consulta y realiza pagos

UI --> Ctrl : envía datos de usuarios,\npagos y solicitudes

Ctrl --> Mod : procesa reglas de negocio

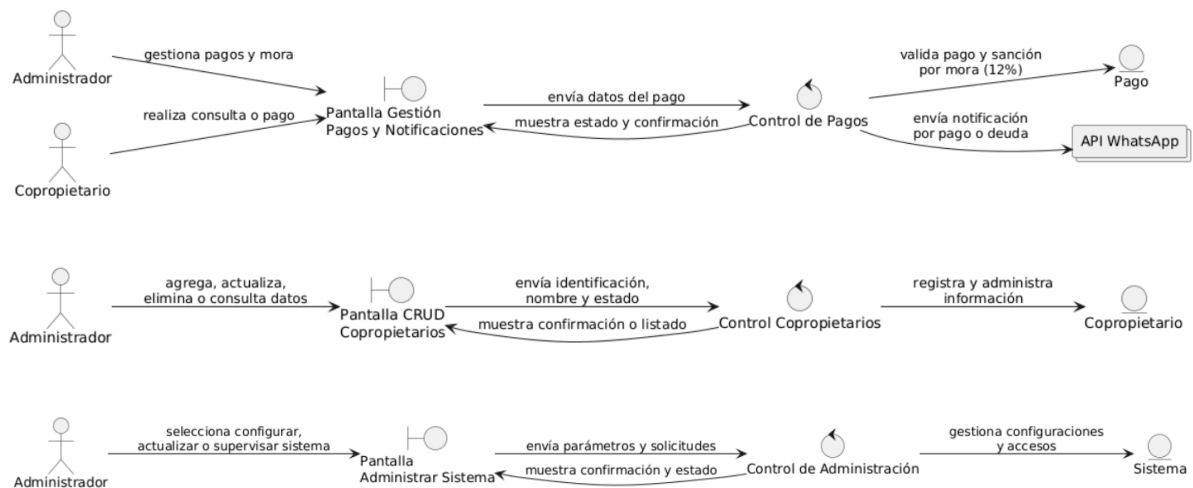
Mod --> DB : registra y consulta información

Mod --> API_WA : envía notificaciones\npor mora y pagos

Ctrl --> UI : muestra confirmaciones,\nreportes y estados

@enduml

Resultado



3. MATRIZ DE TRAZABILIDAD RF-CU

Esta matriz garantiza que todas las interacciones de los pagos y validaciones tengan un requisito asociado y un criterio de aceptación medible.

RF	Caso de uso	Actor	Prioridad	Criterio de aceptación	Evidencia PlantUML
RF-01	Administrar Sistema	Admin, Copro	Alta	El sistema autentica correctamente a los usuarios, restringe el acceso según su rol (RBAC) y permite la recuperación de contraseña vía correo.	UC1
RF-02	Gestionar Copropietarios	Admin, Copro	Alta	El Administrador puede realizar el CRUD y carga masiva (Excel) de residentes. El Copropietario solo puede ver y exportar sus propios reportes de pago.	UC2
RF-03	Implementar Pagos	Copro, Admin	Alta	El flujo se completa cuando el copropietario sube el comprobante, el Administrador valida, y el sistema actualiza saldos (aplicando un 12% extra si hay morosidad).	UC3

RF-04	Enviar Notificaciones	Sistema	Media/Alta	El sistema extrae el número del residente (formato 09...) y envía exitosamente un mensaje automático por WhatsApp al cambiar el estado del pago.	UC4
-------	-----------------------	---------	------------	--	-----

CONCLUSIÓN TÉCNICA

El modelo de casos de uso de Nivel 0 refleja una arquitectura orientada a la separación de responsabilidades y la automatización de procesos críticos. Mediante la implementación de un Control de Acceso Basado en Roles (RBAC) en el módulo Administrar Sistema, se garantiza el principio de mínimo privilegio: los copropietarios interactúan únicamente como emisores de pagos y lectores de sus propios historiales (Gestionar Copropietarios), mientras que el Administrador centraliza el control operativo.

Destaca particularmente el diseño del módulo financiero (Implementar Pagos), el cual encapsula la regla de negocio crítica del condominio (la penalización del 12% por mora). Al vincular este proceso financiero con el módulo de Enviar Notificaciones a través de una relación de dependencia <<include>>, la arquitectura asegura que ningún cambio en el estado de cuenta ocurra de forma silenciosa, utilizando al actor "Sistema" para despachar alertas vía API de WhatsApp. Esto resuelve el objetivo principal del proyecto: reducir la carga operativa del administrador y dotar de total transparencia a las finanzas del conjunto residencial.

RECOMENDACIONES

1. Se recomienda implementar la validación de archivos (JPG/PDF) en el módulo de pagos para evitar la carga de documentos corruptos que afecten la base de datos.
2. Es aconsejable realizar pruebas de integración con la API de WhatsApp en etapas tempranas para asegurar que el flujo de notificaciones no afecte el rendimiento del servidor.

Sangolquí, a 05 de mayo de 2026

ANEXOS (RÚBRICA DE LA GUÍA)

Rúbrica breve de evaluación sobre 20 puntos

Criterio	Descripción	Puntaje	Evidencia
Trazabilidad RF-CU	Relación clara entre requisitos funcionales, actor, caso de uso y criterio de aceptación.	5 pts	Matriz RF-CU completa.
Modelado de casos de uso	Representación correcta de actor, límite del sistema, casos de uso y relaciones.	5 pts	Diagrama PlantUML generado.
Claridad del análisis	Selección pertinente de funcionalidades de nivel 0 y nombres orientados a objetivos del usuario.	4 pts	Casos de uso redactados claramente.
Uso correcto de PlantUML	Código funcional, ordenado y comprensible.	3 pts	Código sin errores de sintaxis.
Presentación técnica	Documento ordenado, con evidencias, explicación breve y coherencia visual.	3 pts	Entrega final documentada.

Formato mínimo de entrega sugerido

Sección	Contenido mínimo	Evidencia
Portada	Datos institucionales, asignatura, NRC, estudiante/equipo, proyecto y fecha.	Documento entregado.
Requisitos funcionales	Listado de RF de nivel 0 para el CRUD de Estudiante.	Tabla RF.
Diagrama de casos de uso	Código PlantUML y captura del diagrama generado.	Código + imagen.
Matriz de trazabilidad	Relación RF, actor, caso de uso, prioridad y criterio de aceptación.	Tabla RF-CU.
Conclusión técnica	Reflexión breve sobre trazabilidad, consistencia y utilidad del modelo.	Texto de cierre.